M3 — Sweeps as one program, optimisers, gradient noise (N1, O3, N2)

jax-experiments writeup · M3

2026-10-17

Contents

N1 — Hyperparameter sweeps as one program **2**
- Question . . . 2
- The idea . . . 2
- The math . . . 2
- What we measure and why . . . 3
- What I built . . . 3
- Results . . . 3
- What surprised me . . . 5
- Open threads . . . 6

O3 — Optimisers beyond Adam **6**

N2 — Gradient noise scale and the critical batch size **6**

Status: N1 done; O3 and N2 not started ·
Hardware: Kaggle T4 ×2 (sm_75, one job per GPU), JAX 0.11.1; CPU (4 cores) for the equivalence tests ·
Data: `data/n1-summary.json` (every number below) and `data/n1/` (one JSON and log per job); ml-ladder's references from its `docs/writeups/data/c-gpu-lr.json` and `c-gpu-lr2-1024.json` ·
Reproduce: `uv run python tools/kaggle/push_r2.py <preset> --push` for `n1-calibrate`, `n1`, `n1-fork` and `n1-fork-torch`, then `uv run python experiments/n1_vmap_sweep/analyze.py`. CPU equivalence: `uv run pytest tests/test_n1_sweep.py`

N1 — Hyperparameter sweeps as one program

Question

ml-ladder's Part C §7 tuned the 4M transformer's learning rate with 18 separate runs. Can one vmapped program train all of them at once, and how much faster is it?

The idea

A sweep is K copies of the same computation. In JAX, training is a pure function `step(params, opt_state, batch, lr)`. Stack K copies of every argument on a leading axis, and `jit(vmap(step))` trains all of them in one compiled program. The learning rate is just another vmapped argument.

So is everything else that differs between the runs:

- each copy's loss scale;
- its skipped steps;
- its batch stream: ml-ladder's runs of one seed share batches, so copies of one seed share one draw per step.

Two questions, kept apart:

- **Is it the same experiment?** Each copy of (lr, seed) starts from ml-ladder's initial weights for that seed and sees that seed's batches, so it can be paired with ml-ladder's run.
- **Is it faster?** Total throughput of K copies in one program, against one copy, which is the rate of K sequential runs.

The math

`vmap` turns every operation on a $[\ldots]$-shaped array into one on a $[K, \ldots]$-shaped array:

- a matmul xW becomes a batched matmul $x_k W_k$;
- per-copy scalars (the loss, the gradient norm used for clipping, the loss-scale decision) become $[K]$-vectors.

Nothing couples the copies, so copy k computes exactly what its sequential run would, up to the order of floating-point reductions.

The speedup has a simple ceiling. If one copy already keeps the GPU's arithmetic units busy, K copies take K times as long. vmap can only win back idle time: kernel-launch overhead, small matmuls that underfill the device, Python between steps.

What we measure and why

- **Equivalence (CPU).** Two seeds × two learning rates, vmapped against four sequential runs. Checked in fp32 and fp16, for a GRU and a transformer.
- **The sweep (T4).** ml-ladder's §7 grid: peak learning rates {3e-4, 6e-4, 1e-3, 2e-3, 4e-3, 8e-3} × seeds 0–2, for the 4M transformer at T=1024. Each run is 3,000 steps of 16 × 1024 bytes, then per-position bits/byte on 262,144 validation bytes. It ran as six programs of three copies.
- **Memory.** XLA's compiled temp memory for K = 1 to 18 copies, with and without per-block remat.
- **Throughput.** Steady tokens/s for K = 1, 2, 4, 8 (and up to 32 for small models), over 200 steps each. Measured on ml-ladder's 4M model and on two much smaller transformers (width 128 and 64, depth 2) at the same 16 × 1024 tokens per step.

What I built

- **`jaxexp.sweep.train_many`:**
  - stacks K copies of parameters, optimiser state and loss scale;
  - runs `jit(vmap(make_step(...)))`;
  - follows each copy's own schedule;
  - draws each unique batch stream once per step.

  It returns one `TrainResult` per copy.
- **`tests/test_n1_sweep.py`:** vmapped copies equal sequential runs.
- **`experiments/n1_vmap_sweep/`:**
  - `run.py`, with `memory`, `throughput` and `sweep` studies;
  - `analyze.py`.
- **The Kaggle runner** now accepts jobs for any experiment script, and gives JAX 92% of the card instead of 75%.

Results

On CPU, vmap computes what the sequential runs compute.

	loss histories	all parameters (relative)	outputs on held-out data
fp32	within 10^{-6}	within 10^{-6}	within 10^{-6}
fp16	within 3×10^{-3}	within 3×10^{-3}	within 3×10^{-3}

Measured, the fp32 parameter differences are 9×10^{-8} to 3×10^{-7}.

One tensor needed explaining: the attention key bias. Per tensor it differs by up to 5% in fp16. Its true gradient is exactly zero, because adding a constant vector to every key shifts all of a query's logits equally, and the softmax ignores that. So its gradient is pure round-off, which Adam scales up to full-size steps in either program. It has no effect on the outputs.

On the T4, the sweep reproduces ml-ladder's curve.

peak LR	ml-ladder bits/byte	JAX, vmapped (3 per program)	largest per-seed difference
3e-4	3.0279 (3.0259–3.0322)	3.0272 (3.0261–3.0312)	0.0010

peak LR	ml-ladder bits/byte	JAX, vmapped (3 per program)	largest per-seed difference
6e-4	2.4353 (2.4041–2.5574)	2.4358 (2.4002–2.5548)	0.0040
1e-3	2.0441 (2.0123–2.2154)	2.0470 (2.0140–2.2166)	0.0029
2e-3	**1.8250** (1.8229–1.8283)	**1.8239** (1.8203–1.8324)	0.0095
4e-3	1.8427 (1.7427–2.0220)	2.0378 (1.7692–2.0624)	0.2197
8e-3	2.1271 (2.0509–2.2613)	2.1067 (2.0571–2.1159)	0.1454

- **The best learning rate is the same: 2e-3**, at 1.824 against 1.825.
- **Up to the optimum, all 12 pairs agree within 0.0095.** At these learning rates the seed-to-seed spread is often wider than that (6e-4: 0.15 in ml-ladder's own runs). So pairing by seed is doing real work: it takes the seed noise out of the comparison.
- **Above the optimum, pairs diverge** by up to 0.22 bits/byte. Yet our ranges overlap ml-ladder's, and its own seed spread there is 0.28 (4e-3) and 0.21 (8e-3). The largest gap, seed 2 at 4e-3, is a fork. Both runs are together through step 750 (2.153 against 2.137). Then ours stalls near 1.67 from step 1,250 to 1,750, while ml-ladder's falls to 1.45.

  A control traced this fork to PyTorch's own run, not to JAX or vmap.

seed 2, lr 4e-3, 3,000 steps	bits/byte
ml-ladder's recorded PyTorch run (2026-10-08)	1.8427
ml-ladder's script, rerun twice (same torch 2.11.0+cu128, T4)	2.0121, 2.0413
JAX, sequential (R2's runner), twice	2.0545, 2.0479
JAX, vmapped group of 3, twice	2.0624, 2.0485

  - **All six fresh runs land together,** PyTorch and JAX, sequential and vmapped (2.01–2.06). The recorded 1.8427 is the outlier: PyTorch took the other branch once and did not reproduce it.
  - **Neither framework's T4 runs are deterministic,** so above the optimum one run per (lr, seed) samples a distribution. Its spread at 4e-3 is at least 0.22 bits/byte.

vmap does not make this sweep faster.

copies K	4M, total tok/s (no remat)	4M, with remat	width-128 transformer	width-64 transformer
1	132.5k (1.00×)	103.2k (1.00×)	385k (1.00×)	436k (1.00×)
2	130.9k (0.99×)	104.4k (1.01×)	370k (0.96×)	456k (1.04×)
4	132.4k (1.00×)	106.1k (1.03×)	402k (1.04×)	473k (1.08×)
8	out of memory	107.6k (1.04×)	400k (1.04×)	484k (1.11×)

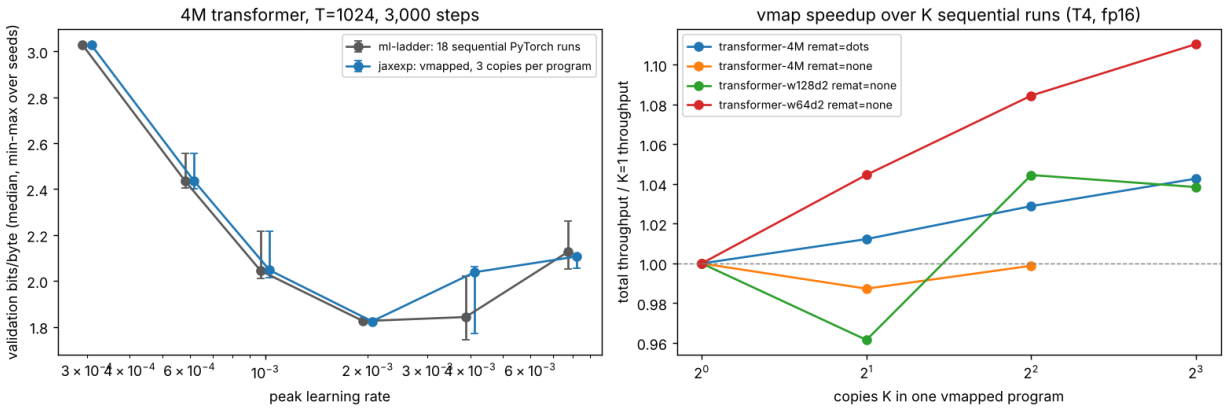


Figure 1: Left: the learning-rate sweep, ours (vmapped) against ml-ladder's sequential runs. Right: throughput of K copies relative to one

- **A single copy already saturates the T4** at 16 × 1024 tokens per step. So K copies take K times as long, and total throughput is flat in K.
- **It stays nearly flat even for a 0.2M-parameter transformer** (1.11× at K=8). With T=1024 and 16 sequences per step, each step's attention and matmuls are large enough to fill the GPU whatever the model width.
- **Memory caps K before speed matters.** XLA's temp memory is 2.87 GiB per copy without remat, so K=4 fits (12.0 GiB) and K=6 does not (15.0 GiB, against 13.8 GiB available). Per-block remat cuts it to 1.26 GiB per copy, so K=8 fits, at 0.78× the speed.
- **The whole sweep cost 1.97 GPU-hours** (six programs of 1,137–1,233 s), against ml-ladder's 1.03 (18 runs × 205 s median). That 0.52× is the same JAX-against-PyTorch gap R2 measured for this model (0.53×), not a cost of vmap.

What vmap *does* buy is that the sweep is one program:

- each group compiles once (9–17 s), not once per run;
- all the copies share one data pipeline;
- the grid is one command and one JSON. N1's sweep tool is now ready for O3 and N2.

What surprised me

PLAN's premise came from the wrong regime. The image probe found that K = 8 copies cost 4.4× one copy, which promised a 1.8× speedup. That probe was a small recurrence whose single copy left the GPU idle. ml-ladder's protocol feeds 16,384 tokens per step, which fills a T4 even with a 0.2M model. The speedup from vmap is the GPU's idle fraction, and here there is almost none.

The attention key bias. Its gradient is exactly zero by symmetry. That showed up as a 5% per-tensor mismatch in what should have been an equality test, and it is invisible in any loss.

The biggest disagreement in the sweep was PyTorch disagreeing with itself. The one pair that looked like a framework difference (0.22 bits/byte) did not survive reruns. ml-ladder's own script, on the same torch version and the same GPU type, landed with JAX (2.01 and 2.04), not at its recorded 1.84. At learning rates above the optimum, runs with identical weights and batches split apart, and nondeterministic GPU arithmetic decides which side a run lands on. The R3 copy task (M2) showed the same thing near its plateau. A single run per configuration in this regime is a draw, not a measurement.

Open threads

- **The small-batch regime.** vmap should pay when one copy cannot fill the device: small batches, short contexts, small models. That is the setting of many hyperparameter studies, and it was not measured here.
- **Deterministic GPU runs** (`--xla_gpu_deterministic_ops`; `torch.use_deterministic_algorithms`), to separate implementation from nondeterminism without repeats.
- **The fork's distribution.** How often does seed 2 at 4e-3 reach about 1.84 rather than about 2.05? Ten repeats would put a number on ml-ladder's luck. The same applies to its 4e-3 entry in §7's table.
- **O3 and N2,** using this sweep tool.

O3 — Optimisers beyond Adam

Not started. See `PLAN.md`.

N2 — Gradient noise scale and the critical batch size

Not started. See `PLAN.md`.